

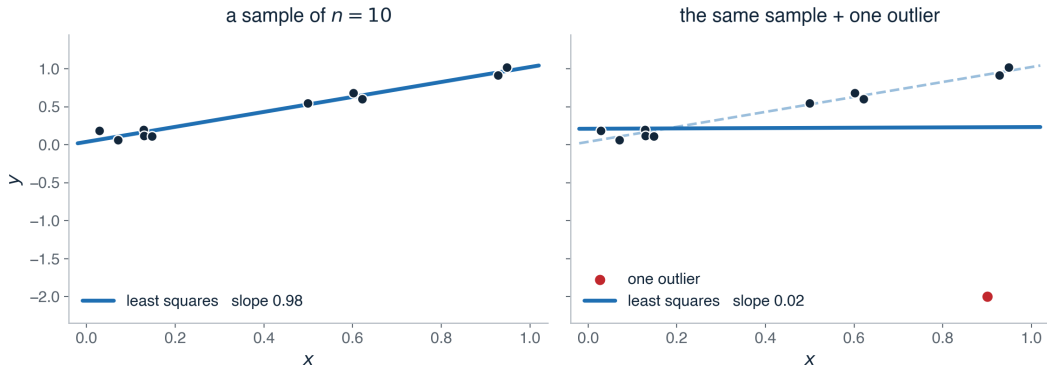
Outliers and the ℓ_1 Loss

MATH 4432 Statistical Machine Learning · Tutorial T04

Department of Mathematics, HKUST

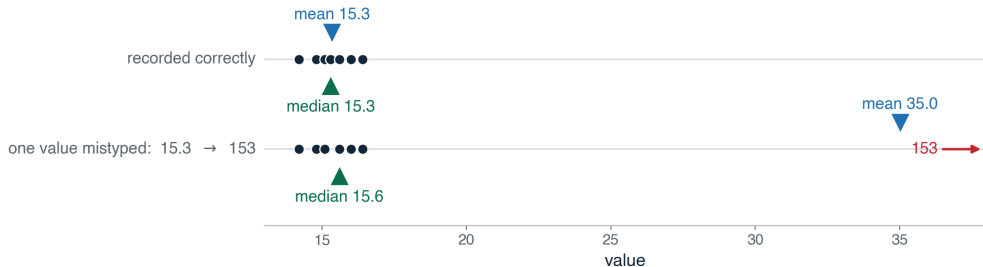
Fall 2026

One observation can move the fit



one outlier among 11 observations changes the least-squares slope from 0.98 to 0.02

Mean and median



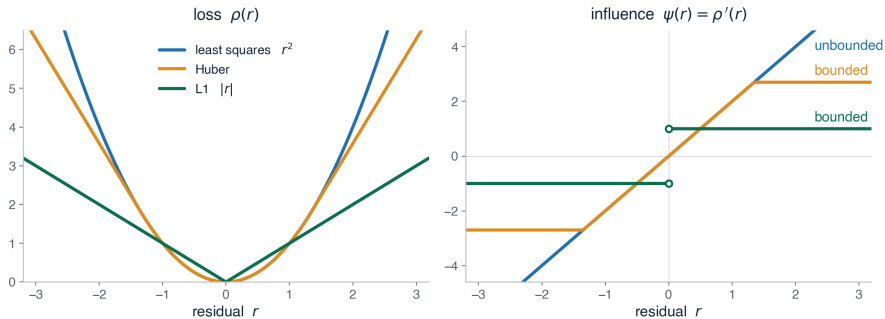
mean $\bar{y} = \arg \min_c \sum_{i=1}^n (y_i - c)^2$

median $\text{med}(y) = \arg \min_c \sum_{i=1}^n |y_i - c|$

$$y_i = x_i^\top \beta + \varepsilon_i, \quad r_i(\beta) = y_i - x_i^\top \beta, \quad \hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n \rho(r_i(\beta))$$

- $\rho(r) = r^2$ **least squares** $\sum_i r_i^2 = \|y - X\beta\|_2^2$
- $\rho(r) = |r|$ **least absolute deviations (L1)** $\sum_i |r_i| = \|y - X\beta\|_1$
- **outlier**: an observation whose noise is far larger than that of the rest of the sample

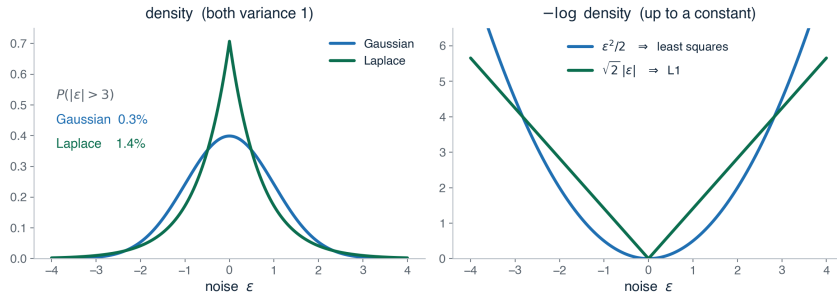
Loss and influence



$$\sum_{i=1}^n \psi(r_i(\hat{\beta})) x_i = 0$$

each observation pulls on $\hat{\beta}$ with force $\psi(r_i)$

Each loss assumes a noise distribution



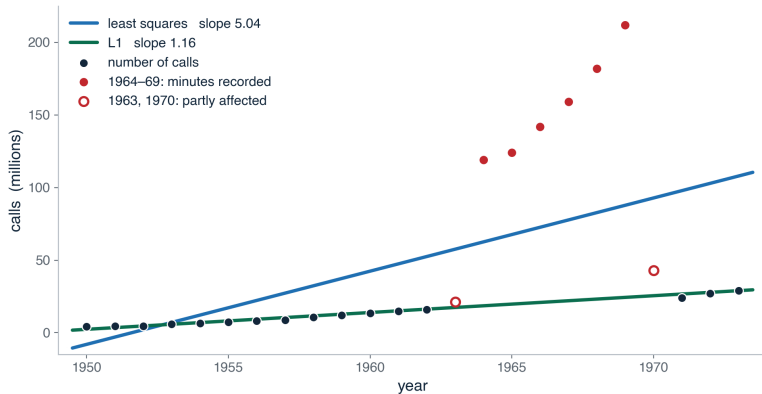
$$\arg \max_{\beta} \prod_{i=1}^n p(r_i(\beta)) = \arg \min_{\beta} \sum_{i=1}^n \underbrace{-\log p(r_i(\beta))}_{\rho(r_i)}$$

Three losses

	least squares	Huber	L1
$\rho(r)$	r^2	quadratic, then linear	$ r $
influence $\psi(r)$	unbounded	bounded	bounded
noise model	Gaussian	Gaussian centre, Laplace tails	Laplace
efficiency, Gaussian noise	100%	95%	64%
outliers in y	not robust	robust	robust
outliers in x	not robust	not robust	not robust
solution	closed form	iterative	iterative / LP

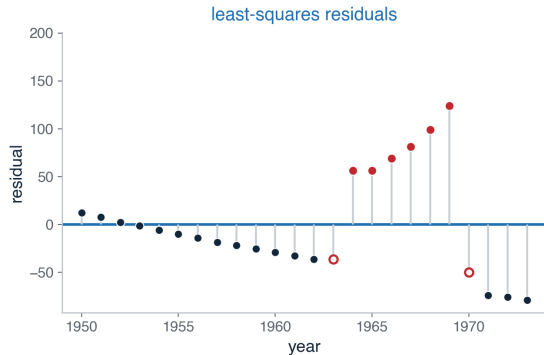
Huber with $c = 1.345 \sigma$; efficiency = $\text{Var}(\hat{\beta}_{\text{LS}})/\text{Var}(\hat{\beta})$ as $n \rightarrow \infty$

Belgian phone calls, 1950–1973

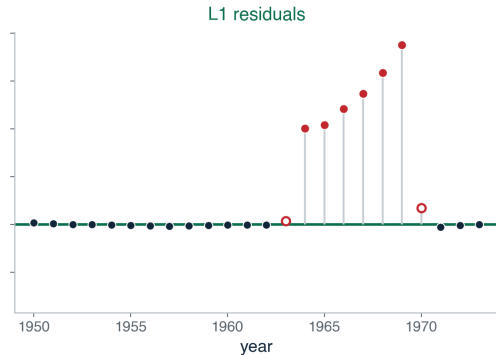


1964–1969: total minutes of calls recorded instead of the number of calls (Rousseeuw & Leroy, 1987)

Residuals



a clean year (1973) has residual -79 ,
an outlier (1964) only $+56$

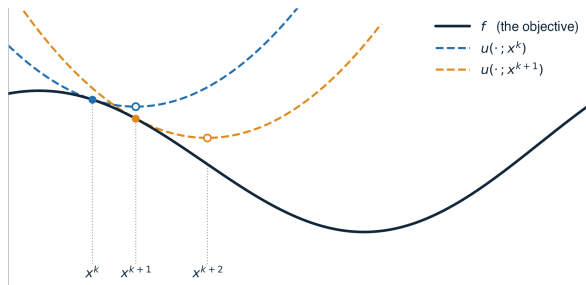


clean years within ± 2.7 ,
every outlier above $+100$

$$\text{L2: } \hat{\beta} = (X^T X)^{-1} X^T y \qquad \text{L1: } f(\beta) = \sum_{i=1}^n |y_i - x_i^T \beta|$$

- f is convex, but not differentiable where some $r_i = 0$
- **majorization-minimization** $\Rightarrow$ iteratively reweighted least squares (IRLS)
- **subgradient method**
- **linear programming** exact; used by `sklearn`

Majorization–minimization (MM)



majorizer $u(\cdot; x^k)$ of f :

$$u(x; x^k) \geq f(x) \quad \text{for all } x$$

$$u(x^k; x^k) = f(x^k)$$

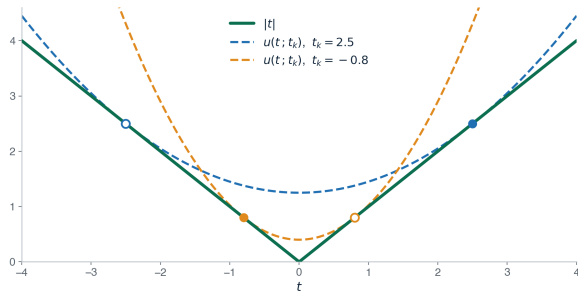
update

$$x^{k+1} = \arg \min_x u(x; x^k)$$

descent

$$\begin{aligned} f(x^{k+1}) &\leq u(x^{k+1}; x^k) \\ &\leq u(x^k; x^k) = f(x^k) \end{aligned}$$

A quadratic majorizer of $|t|$



$$u(t; t_k) = \frac{t^2 + t_k^2}{2|t_k|}$$

$$u(t; t_k) - |t| = \frac{(|t| - |t_k|)^2}{2|t_k|} \geq 0$$

equality at $|t| = |t_k|$

quadratic in t : easy to minimise

Apply the majorizer to every residual, at $t_k = r_i(\beta^k)$:

$$\sum_{i=1}^n |r_i(\beta)| \leq \sum_{i=1}^n \frac{r_i(\beta)^2 + r_i(\beta^k)^2}{2 |r_i(\beta^k)|}$$

$$\beta^{k+1} = \arg \min_{\beta} \sum_{i=1}^n w_i^k r_i(\beta)^2 = (X^T W^k X)^{-1} X^T W^k y, \quad w_i^k = \frac{1}{|r_i(\beta^k)|}$$

- every step is a **weighted least-squares** fit
- large residual $\Rightarrow$ small weight

Subgradient method

$$\partial|t| = \begin{cases} \{\text{sign}(t)\} & t \neq 0 \\ [-1, 1] & t = 0 \end{cases} \quad \Rightarrow \quad g^k = -X^T \text{sign}(y - X\beta^k) \in \partial f(\beta^k)$$

$$\beta^{k+1} = \beta^k - \alpha_k g^k$$

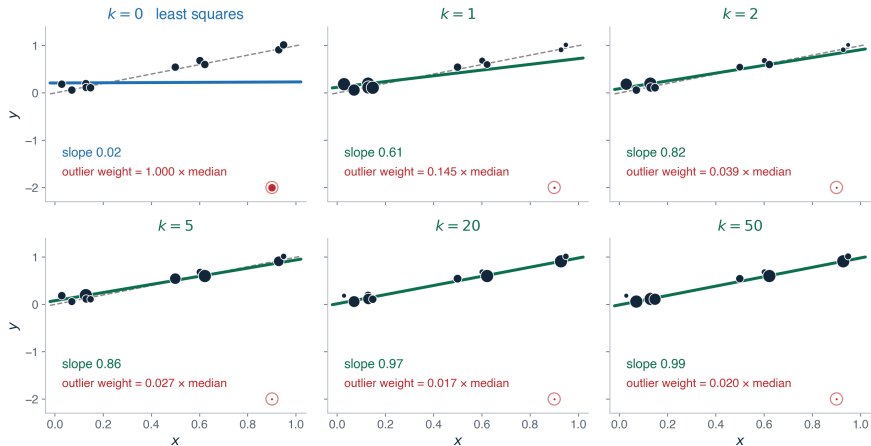
- not a descent method: $f(\beta^{k+1}) > f(\beta^k)$ can happen
- needs $\alpha_k \rightarrow 0$ and $\sum_k \alpha_k = \infty$, e.g. $\alpha_k = \alpha_0 / \sqrt{k+1}$
- keep the best iterate

1 · IRLS

```
def irls_l1(X, y, n_iter=50, delta=1e-10):
    beta = np.linalg.lstsq(X, y, rcond=None)[0]      # start: least squares
    for k in range(n_iter):
        r = y - X @ beta
        w = 1 / np.maximum(np.abs(r), delta)         # w_i = 1/|r_i|, guarded
        Xw = X * w[:, None]
        beta = np.linalg.solve(X.T @ Xw, Xw.T @ y)  # weighted least squares
    return beta
```

- delta guards against $r_i = 0$
- `sm.WLS(y, X, weights=w)` minimises $\sum_i w_i r_i^2$: pass w , not $\sqrt{w}$ ($\sqrt{w}$ gives the $\ell_{1.5}$ loss)

IRLS on the example

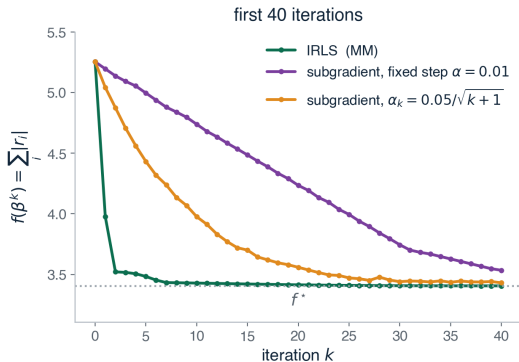
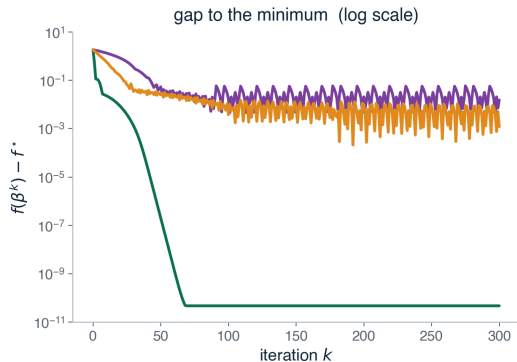


marker area $\propto$ the weight of each observation

2 · subgradient method

```
def subgradient_l1(X, y, step, n_iter=300):  
    l1 = lambda b: np.abs(y - X @ b).sum()  
    beta = np.linalg.lstsq(X, y, rcond=None)[0]  
    best = beta.copy()  
    for k in range(n_iter):  
        g = -X.T @ np.sign(y - X @ beta)           # a subgradient of f  
        beta = beta - step(k) * g  
        if l1(beta) < l1(best):                       # not a descent method:  
            best = beta.copy()                        # keep the best iterate  
    return best  
  
step = lambda k: 0.05 / np.sqrt(k + 1)             # diminishing step size
```

Convergence



IRLS: gap 2×10^{-7} after 50 iterations · subgradient, $\alpha_k = 0.05/\sqrt{k+1}$: gap 10^{-3} after 300, and f increases on 116 of them

3 · real data, and an exact solver

```
from sklearn.linear_model import QuantileRegressor, HuberRegressor
x, y = phones.year.values - 1950, phones.calls.values
X = np.column_stack([np.ones_like(x), x])
b_irls = irls_l1(X, y)
lad = QuantileRegressor(quantile=0.5, alpha=0, solver="highs").fit(x[:, None], y)
hub = HuberRegressor(epsilon=1.35, alpha=0).fit(x[:, None], y)
```

	slope	$\sum_i r_i $
least squares	5.04	1018.9
Huber	1.92	850.4
L1, IRLS	1.56	844.0
L1, linear programming	1.16	844.0
least squares, clean years only	1.08	850.0

same loss, different lines:

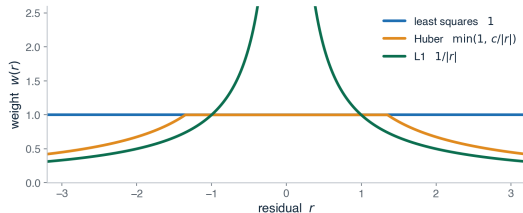
every slope in [1.16, 1.62]
attains the minimum

the L1 solution need not be
unique

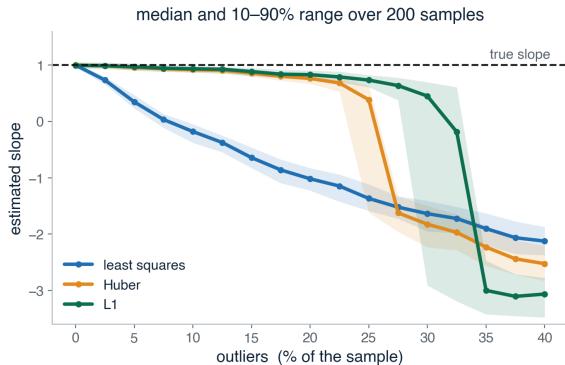
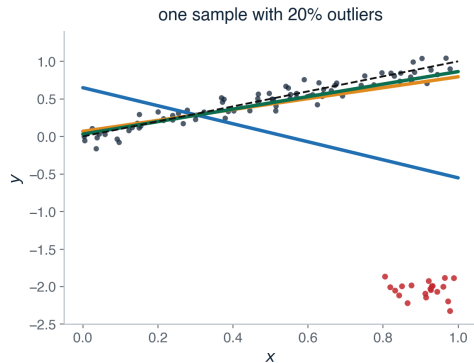
4 · one loop, three losses

$$w(r) = \frac{\psi(r)}{r}$$

```
w = np.ones_like(r)           # least squares  
w = np.minimum(1, c / np.abs(r)) # Huber  
w = 1 / np.maximum(np.abs(r), delta) # L1
```

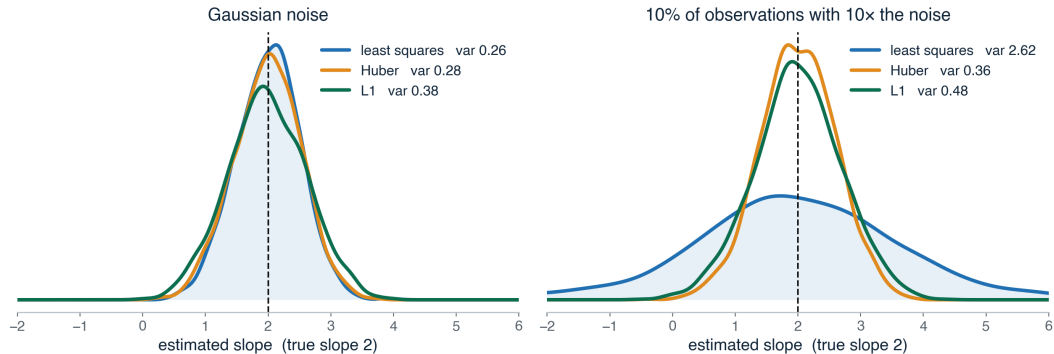


5 · more and more outliers



$n = 100$ from $y = x + \varepsilon$; a fraction of the observations replaced by outliers at $x \in [0.8, 1]$,
 $y \approx -2$

6 · robustness has a price

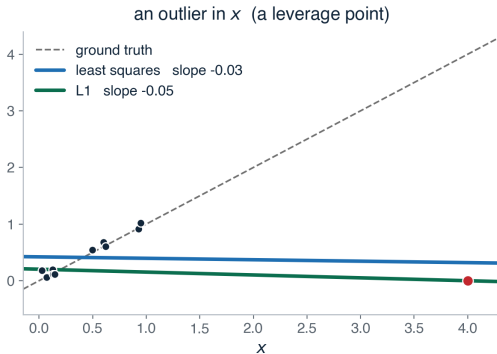
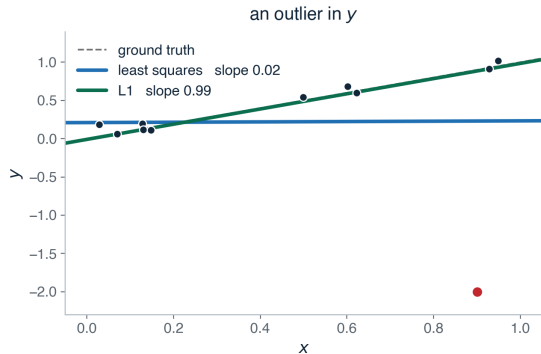


$n = 50$, 2000 samples · variance of the L1 slope: $\times 1.49$ without outliers, $\times 0.18$ with outliers

Take away

- the loss decides how far one observation can pull the fit: unbounded ψ vs bounded ψ
- squared loss $\leftrightarrow$ mean $\leftrightarrow$ Gaussian noise; absolute loss $\leftrightarrow$ median $\leftrightarrow$ Laplace noise
- no closed form for L1: MM turns it into a sequence of weighted least-squares fits
- robustness costs efficiency when there are no outliers; Huber is the compromise
- look for outliers in the residuals of a robust fit

Bonus: outliers in x



L1 bounds the influence of r_i , not of x_i · for leverage points: least trimmed squares, MM-estimators, RANSAC

- Rousseeuw & Leroy (1987), *Robust Regression and Outlier Detection* — the phones data
- Huber & Ronchetti (2009), *Robust Statistics* · ESL §10.6
- Hunter & Lange (2004), A tutorial on MM algorithms, *Amer. Statistician*
- Boyd, EE364b notes — subgradient methods · Palomar, ELEC 5470 — MM
- T04_OutliersL1.ipynb — every experiment on these slides